SYSTEM DESIGN

LSM Trees Visual Guide

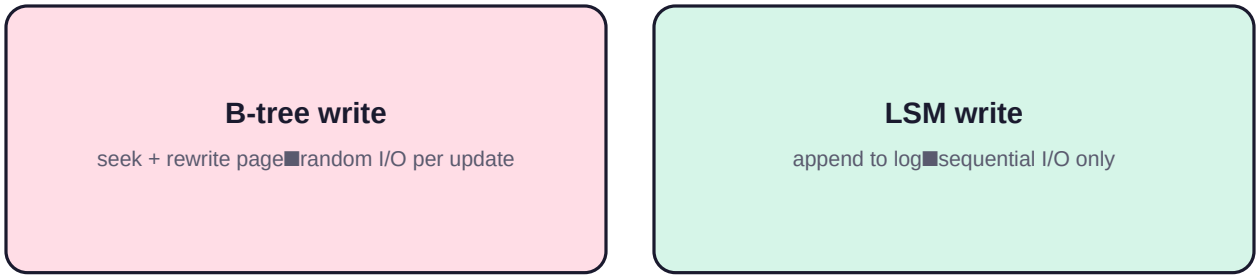
Log-Structured Merge trees:
turn random writes into sequential ones.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Write Problem

B-trees do random disk writes per update -- disks (and SSDs) hate random writes.

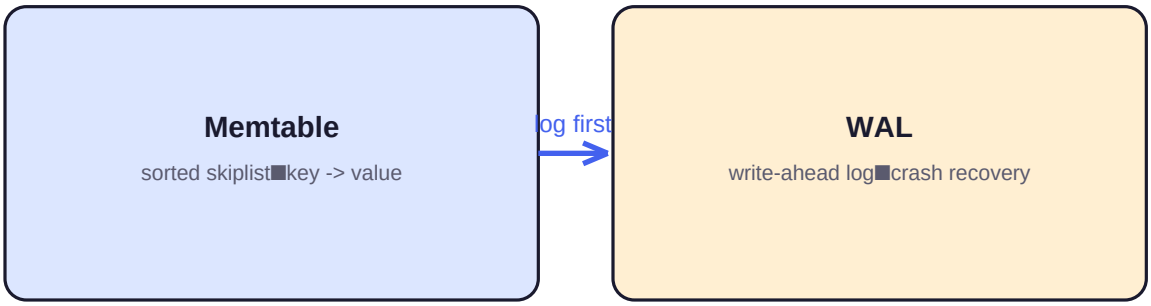


- Sequential writes are 10-100x faster than random ones on spinning disks -- and kinder to SSDs.
- LSM insight: buffer writes in memory, then flush them to disk as big SORTED chunks.
- Trade-off accepted: reads get harder (check several places) to make writes blazing fast.

STEP 2/6

The Memtable

An in-memory sorted structure (skiplist): all writes land here first.

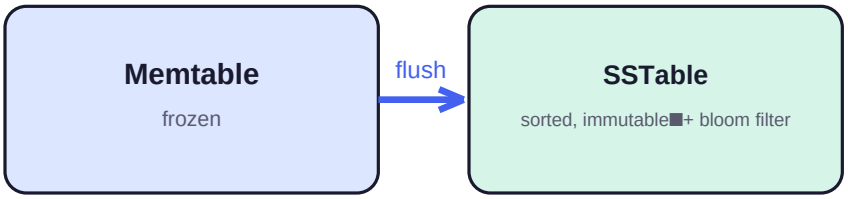


- Write path: append to WAL (durability), insert into memtable (speed). Both are sequential/in-memory.
- Reads check the memtable FIRST -- the freshest data lives here.
- When the memtable fills (e.g. 64MB), it freezes and a new empty one takes over.

STEP 3/6

Immutable SSTables

Frozen memtables flush to disk as Sorted String Tables -- never modified again.

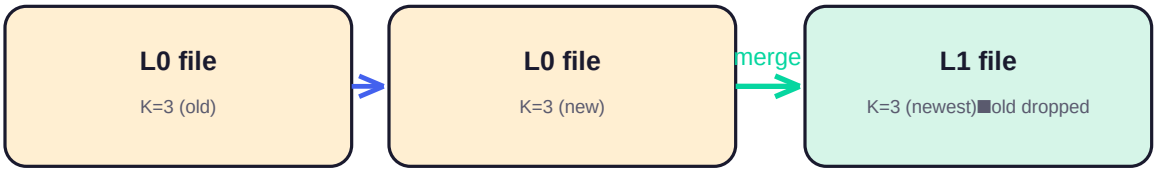


- Immutability is the superpower: no locks, no torn writes, trivially cacheable and shareable.
- Each SSTable carries an index + bloom filter: 'is key K in this file?' without reading it.
- Deletes are tombstones: a marker 'K deleted' that shadows older copies until compaction.

STEP 4/6

Compaction

Merge SSTables, drop overwritten keys and tombstones, keep levels tidy.

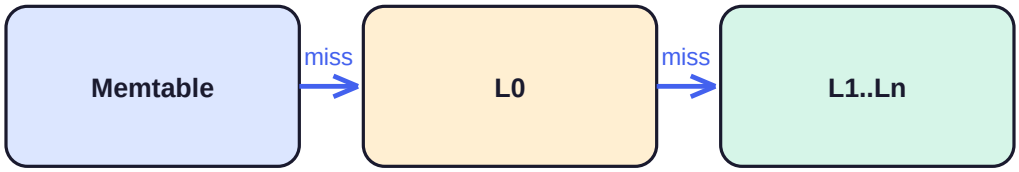


- Levels grow 10x per level (L0 -> L1 -> L2): deeper levels hold older, larger, fully-sorted runs.
- Compaction is background I/O: it costs write amplification (rewriting data) to buy read speed.
- Leveled (RocksDB) vs tiered (Cassandra STCS): fewer, bigger merges vs many small ones.

STEP 5/6

The Read Path

Newest first: memtable, then L0, then deeper levels -- first hit wins.



- Tombstones checked at every level: a delete marker shadows the old value until compacted away.
- Bloom filters short-circuit most levels: 'definitely not here' without touching disk.
- Read amplification is the LSM tax: one logical read may probe several files.

STEP 6/6

LevelDB and RocksDB

The engines that made LSM the default for write-heavy stores.

LevelDB

Google's embeddable LSM (2011) -- the reference design

RocksDB

Meta's fork: tuned for SSDs, multi-threaded compaction, column families

Cassandra / ScyllaDB

LSM per node + distributed layer -- planet-scale writes

CockroachDB (Pebble)

RocksDB-inspired Go engine under a distributed SQL layer

STEP ✓I✓

Key Takeaways

LSM Trees — Remember

- LSMs turn random writes into sequential ones: buffer in a memtable, flush sorted SSTables.
- WAL first for durability; frozen memtables flush immutable -- locks disappear from the write path.
- Compaction merges levels, drops overwritten keys and tombstones, at the cost of write amplification.
- Reads go newest-first (memtable -> L0 -> deeper), using bloom filters to skip absent keys.
- RocksDB/LevelDB/Cassandra prove it: for write-heavy workloads, LSM beats B-trees.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev